Assessment Tool 1: Artificial Intelligence (CSA17)

Oct 14, 2026 · @Vignesh

This submission by Vignesh SH (Reg. No. 192412046) answers all five questions of Annexure A, Analytical Problem Solving, for Dr K Rajagopal at SIMATS Engineering.

Submission details

Item	Details
Institution	SIMATS Engineering
Assessment	Assessment Tool 1, Annexure A: Analytical Problem Solving
Course	Artificial Intelligence (Course Code: CSA17)
Course outcome covered	CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively (BL3)
Weightage, marks, duration	35% weightage, 5 questions, 50 marks, 1 hour
Student name	Vignesh SH
Register number	192412046
Course faculty	Dr K Rajagopal
Date of submission	14 October 2026

Code of conduct

I, Vignesh SH (Reg. No. 192412046), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Question 1: Water Jug Problem

Exactly 2 gallons can be measured into the 4-gallon jug in 6 operations, and a breadth-first search over the state space confirms that no shorter sequence exists.

Problem formulation

- **State:** an ordered pair (x, y), where x is the water in the 4-gallon jug (0 to 4) and y is the water in the 3-gallon jug (0 to 3).
- **Initial state:** (0, 0), both jugs empty.

- **Goal state:** (2, n) for any n, that is, exactly 2 gallons in the 4-gallon jug.
- **Path cost:** one unit per operation, so the best solution uses the fewest operations.

Production rules (actions)

Rule	Condition	Action	Result
R1	$x < 4$	Fill the 4-gallon jug from the pump	(4, y)
R2	$y < 3$	Fill the 3-gallon jug from the pump	(x, 3)
R3	$x > 0$	Empty the 4-gallon jug on the ground	(0, y)
R4	$y > 0$	Empty the 3-gallon jug on the ground	(x, 0)
R5	$x + y \geq 4$ and $y > 0$	Pour from the 3-gallon jug into the 4-gallon jug until it is full	$(4, y - (4 - x))$
R6	$x + y \geq 3$ and $x > 0$	Pour from the 4-gallon jug into the 3-gallon jug until it is full	$(x - (3 - y), 3)$
R7	$x + y \leq 4$ and $y > 0$	Pour all of the 3-gallon jug into the 4-gallon jug	(x + y, 0)
R8	$x + y \leq 3$ and $x > 0$	Pour all of the 4-gallon jug into the 3-gallon jug	(0, x + y)

Solution path

Step	State (x, y)	Rule applied	What happens
0	(0, 0)	Start	Both jugs are empty
1	(0, 3)	R2	Fill the 3-gallon jug from the pump
2	(3, 0)	R7	Pour all 3 gallons into the 4-gallon jug
3	(3, 3)	R2	Fill the 3-gallon jug again
4	(4, 2)	R5	Pour 1 gallon into the 4-gallon jug until it is full; 2 gallons remain in

Step	State (x, y)	Rule applied	What happens
			the 3-gallon jug
5	(0, 2)	R3	Empty the 4-gallon jug on the ground
6	(2, 0)	R7	Pour the 2 gallons into the 4-gallon jug

Result: the 4-gallon jug holds exactly 2 gallons and the 3-gallon jug is empty. A second 6-step route also works: (0, 0), (4, 0), (1, 3), (1, 0), (0, 1), (4, 1), (2, 3). Breadth-first search explores states level by level, so the first level where a goal state appears (level 6) is the optimal number of operations.

Question 2: Mars Rover Agent

A Mars Rover is best modelled as a hybrid, layered, utility-based agent working in a partially observable, dynamic and continuous environment. The answers below follow the PEAS framework (Performance, Environment, Actuators, Sensors).

i. Percepts

The percepts are the inputs the rover receives through its sensors:

- Stereo, panoramic and hazard-avoidance camera images of terrain, rocks and horizon
- Spectrometer and X-ray readings of the chemical and mineral content of soil and rock
- Wheel encoder and odometry readings, plus inclination (tilt) and gyroscope data
- Temperature, wind, dust and atmospheric sensor readings
- Battery charge, solar panel output, motor current and internal health status
- Drill, arm and wheel torque feedback (for example, a wheel slipping or an obstacle blocking motion)
- Commands and updated targets uplinked from the Earth control centre, and communication link status

ii. Operating environment

Property	Classification	Reason
Observability	Partially observable	Cameras and sensors see only a limited range; terrain beyond the horizon and below the surface is unknown
Determinism	Stochastic	Wheel slip, loose sand, dust and sensor noise mean an action does not always give the expected result
Episodic or sequential	Sequential	Current moves affect later options, such as battery use,

Property	Classification	Reason
		position and wear
Static or dynamic	Dynamic	Dust storms, temperature swings and lighting change while the rover is deciding
Discrete or continuous	Continuous	Position, speed, temperature and orientation take continuous values
Agents	Single agent, with remote supervision	No competing agents; Earth operators only send high-level goals
Known or unknown	Largely unknown	The terrain and the science targets must be discovered by exploring

Two further constraints shape the design: the one-way radio delay to Earth is several minutes (roughly 3 to 22 minutes depending on planetary positions), so the rover cannot be joystick-driven, and power and communication windows are limited.

iii. Actions

- Move forward or backward, steer, turn, stop
- Plan a path and avoid or climb around obstacles
- Point the camera mast and capture images
- Extend the robotic arm, scrape, drill or abrade rock and collect a sample
- Place instruments on a target and analyse the sample
- Store, compress and transmit data to Earth; receive commands
- Manage power: charge from solar panels, sleep, heat or cool components
- Run self-diagnosis and enter safe mode on a fault

iv. Performance measure

- Number and scientific value of samples correctly analysed
- Area explored and distance covered safely
- Quantity and quality of data successfully transmitted to Earth
- Safety: no tipping over, getting stuck or damaging instruments
- Energy efficiency and mission lifetime
- Time taken to reach and complete each target

These criteria conflict (for example, speed against safety and energy), so a weighted combination of them is used as the overall score.

v. Suitable agent architecture

The most suitable choice is a **hybrid (layered) goal-based and utility-based agent with a reactive safety layer and a learning component.**

- **Reactive layer:** immediate rules such as stop when tilt is too high or an obstacle is detected. Radio delay makes waiting for Earth unsafe, so these reflexes run on board.
- **Goal-based layer:** the rover keeps an internal model of the map and its own state (needed because the environment is only partially observable) and plans a route to each science target.
- **Utility-based layer:** it chooses between competing options, such as a nearer but less interesting rock versus a distant, more valuable one, by trading science value against energy, risk and time.
- **Learning element:** it improves terrain classification and target selection from past experience.

A simple reflex agent is unsuitable because it has no memory or goals, and a purely goal-based agent cannot weigh trade-offs, so the layered design is the best fit.

Question 3: 8-Queens Problem

The best formulation places one queen per column, adding queens only on squares that are not attacked, which cuts the search space from about 1.8 × 10^14 sequences to just 2,057 states, and depth-first search with backtracking then finds a solution quickly.

Why the configuration in the question fails

Two queens attack each other if they share a row, a column or a diagonal. For queens in columns i and j at rows r_i and r_j, a diagonal attack exists when $|r_i - r_j| = |i - j|$. A queen in the first column and a queen in the last column ($|i - j| = 7$) therefore attack along the long diagonal when their rows differ by 7, which is why the shown configuration is not a solution.

Problem formulation

Component	Naive incremental formulation	Improved formulation (recommended)
States	Any arrangement of 0 to 8 queens on the board	Any arrangement of n queens ($0 \le n \le 8$), one per column in the leftmost n columns, none attacking another
Initial state	Empty board	Empty board
Actions	Add a queen to any empty square	Add a queen to a non-attacked square in the leftmost empty column
Transition model	Returns the board with the new queen added	Same
Goal test	8 queens on the board, none attacked	Same

Component	Naive incremental formulation	Improved formulation (recommended)
Path cost	Not important (every solution has 8 placements); 1 per step	Same
Size of search space	64 × 63 × … × 57 ≈ 1.8 × 10^14 sequences	2,057 states

A **complete-state formulation** is also possible: start with all 8 queens placed, one per column (8^8 = 16,777,216 states), and move a queen within its column to reduce attacks. That suits local search, but the question asks about uninformed search, so the incremental form is used.

Exploring the search space with search strategies

Strategy	Behaviour on 8-Queens	Suitability
Breadth-first search	Expands every partial placement level by level; all solutions lie at depth 8, so it finds one only after storing very large levels	Poor: heavy memory use
Depth-first search with backtracking	Places a queen, goes deeper, and backs up as soon as no safe square is left; stores only the current path of at most 8 queens	Best: low memory, goals all at the same depth
Depth-limited search (limit 8)	Same as DFS but guaranteed to stop at depth 8	Good
Uniform-cost search	All steps cost 1, so it behaves like breadth-first search	Poor

Depth-first search with backtracking, trying rows 1 to 8 in each column, produces the first solution (rows by column) 1, 5, 8, 6, 3, 7, 2, 4. In total there are 92 solutions, of which 12 are distinct once rotations and reflections are treated as the same.

A valid solution (Q = queen; columns 1 to 8 left to right)

Row	C1	C2	C3	C4	C5	C6	C7	C8
1	Q	·	·	·	·	·	·	·
2	·	·	·	·	·	·	Q	·
3	·	·	·	·	Q	·	·	·
4	·	·	·	·	·	·	·	Q

Row	C1	C2	C3	C4	C5	C6	C7	C8
5	·	Q	·	·	·	·	·	·
6	·	·	·	Q	·	·	·	·
7	·	·	·	·	·	Q	·	·
8	·	·	Q	·	·	·	·	·

Every row, column and diagonal holds at most one queen, so the goal test is satisfied.

Question 4: OLA Cab Booking Agent

The suitable agent is a **goal-based problem-solving agent with a utility-based choice of cab type**: it has a clear goal (reach the destination), searches for a route, and ranks the cab options by how well they match the customer's comfort, cost and time preferences.

Why this agent type

- The customer states a goal (a destination), so the agent formulates a goal and a problem, then searches for a sequence of actions that reaches it.
- Several cab types (mini, micro, sedan, shared, prime and so on) can reach the goal, so a simple reflex agent is not enough. A utility function scores each option on fare, waiting time, travel time and comfort, and the agent picks the highest score.
- The environment is fully observable for the app (map, prices and driver locations are known), deterministic for planning purposes, sequential and dynamic (traffic and availability change), so the agent re-plans when conditions change.

Problem formulation

Component	Definition
Initial state	Customer at the pickup location, no cab booked
Goal test	Customer is at the destination
Actions	Select cab type, confirm booking, driver moves along a road segment, start trip, end trip
Transition model	Applying an action gives the new location and booking status
Path cost	Fare, travel time and waiting time (weighted)

Pseudocode

```
function OLA-PROBLEM-SOLVING-AGENT(percept) returns an action
    persistent: seq      // action sequence, initially empty
                state    // description of the current world state
                goal     // destination, initially null
                problem  // problem formulation
```

```
    state <- UPDATE-STATE(state, percept)   // location, traffic, available cabs,
prices

    if seq is empty then
        goal    <- FORMULATE-GOAL(state)    // destination entered by the customer
        problem <- FORMULATE-PROBLEM(state, goal)

        // Step 1: find all cab types available at the pickup location
        options <- GET-AVAILABLE-CABS(state)    // mini, micro, sedan, shared, prime,
...

        // Step 2: choose the cab type with the highest utility
        best <- null
        bestScore <- -infinity
        for each cab in options do
            fare    <- ESTIMATE-FARE(cab, state, goal)
            wait    <- ESTIMATE-ARRIVAL-TIME(cab, state)
            travel  <- ESTIMATE-TRIP-TIME(cab, state, goal)
            score   <- UTILITY(fare, wait, travel, COMFORT(cab), customer.preference)
            if score > bestScore then
                best <- cab
                bestScore <- score

        // Step 3: search for the best route to the destination
        route <- SEARCH(problem)                // UCS or A* on the road network
        if route = failure then return "No cab or route available"

        seq <- [CONFIRM-BOOKING(best), ASSIGN-DRIVER(best), PICKUP, route, END-TRIP,
PAYMENT]

    action <- FIRST(seq)
    seq    <- REST(seq)

    if percept shows traffic change or cab cancellation then
        seq <- []                               // re-plan on the next call

    return action
```

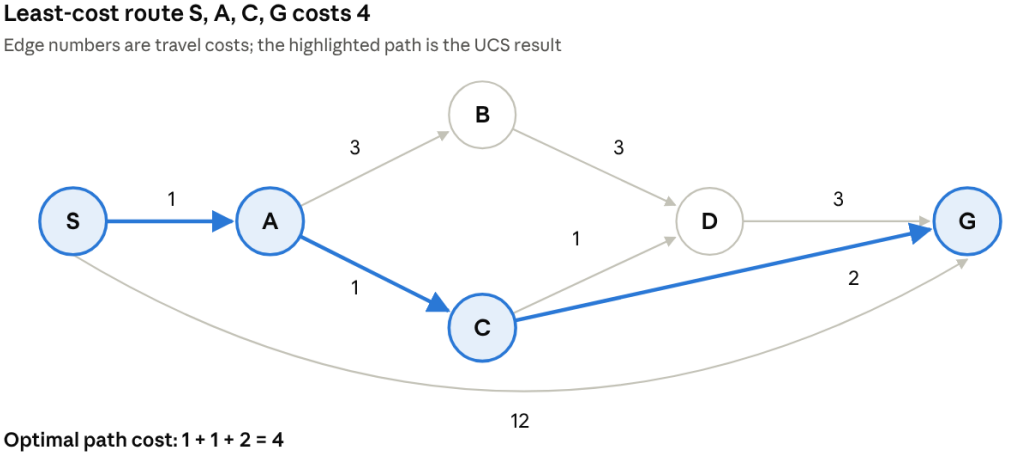
The agent repeats the cycle of update state, formulate goal, formulate problem, search and execute, so a cancelled booking or heavy traffic triggers a fresh plan.

Question 5: Uniform Cost Search on the Delivery Network

Uniform Cost Search (UCS) finds the least-cost route S → A → C → G with a total cost of 4, expanding six nodes in the order S, A, C, D, B, G.

Problem understanding

The network is a directed weighted graph with edge costs S→A = 1, S→G = 12, A→B = 3, A→C = 1, B→D = 3, C→D = 1, C→G = 2 and D→G = 3. The start is warehouse S, the goal is warehouse G, and the cost to minimise is the sum of route costs (fuel plus time).



delivery network · 6 warehouses, 8 directed routes

The highlighted edges mark the least-cost path found by UCS.

Method and justification

UCS keeps a priority queue (frontier) ordered by the path cost g(n) from S and always expands the cheapest node first. The goal test is applied when a node is removed from the queue, not when it is generated, which guarantees the first goal found is optimal. UCS is the right method here because the route costs are unequal and non-negative, so breadth-first search would not minimise cost. Ties are broken in the order the nodes were added to the frontier.

Step-by-step trace

Step	Node expanded (g)	Nodes added or updated	Frontier after the step (node: g)
1	S (0)	A = 0 + 1 = 1; G = 0 + 12 = 12	A: 1, G: 12
2	A (1)	B = 1 + 3 = 4; C = 1 + 1 = 2	C: 2, B: 4, G: 12
3	C (2)	D = 2 + 1 = 3; G = 2 + 2 = 4, which improves on 12	D: 3, B: 4, G: 4
4	D (3)	G = 3 + 3 = 6, which is worse than 4, so it is ignored	B: 4, G: 4
5	B (4)	D = 4 + 3 = 7, which is worse than 3, so it is ignored	G: 4

Step	Node expanded (g)	Nodes added or updated	Frontier after the step (node: g)
6	G (4)	Goal test passes, so the search stops	Empty

All routes from S to G, for comparison

Route	Total cost
S → A → C → G (optimal)	1 + 1 + 2 = 4
S → A → C → D → G	1 + 1 + 1 + 3 = 6
S → A → B → D → G	1 + 3 + 3 + 3 = 10
S → G (direct)	12

Interpretation of results

The delivery route with the lowest cost is S → A → C → G at a total cost of 4 cost units (fuel plus time). The direct route S → G costs three times as much (12), so the company saves 8 cost units per delivery by going through warehouses A and C. UCS is complete and optimal for non-negative costs; its time and memory grow with the number of nodes cheaper than the optimal cost, and here only six nodes were expanded.